Variational Autoencoders

ELG 5218 - Uncertainty Evaluation in Engineering Measurements and Machine Learning

Adapted from Philip Schulz and Wilker Aziz https://github.com/vitutorial/VITutorial

University of Ottawa

November 9, 2026

Roadmap

1. Parametrizing Distributions with Neural Networks
2. Autoencoders as deterministic representation learning
3. Variational Autoencoders
   - VAE Introduction
4. VAE Derivation
5. Using the Trained VAE
6. Case Study: Breathing Anomaly Detection
7. Practical Issues and Recommendations

Why Neural Networks for Probabilistic Models?

Neural Networks alone

- Excel at supervised tasks
- Struggle with *lack of supervision*
- No built-in uncertainty

Probabilistic Models alone

- Principled uncertainty
- Limited expressiveness
- Hand-crafted features

The marriage: Deep Generative Models

Let neural networks **parametrize the distributions** of a probabilistic model – we get the best of both worlds.

Distributions Parametrized by NNs

Idea: keep a standard distribution, but let a neural network compute its parameters from side information c (inputs, context).

	Discrete $X \in \{1, \dots, K\}$	**Continuous** $X \in \mathbb{R}$
Classical	$X \sim \mathrm{Cat}(\pi_1, \dots, \pi_K)$	$X \sim \mathcal{N}(\mu, \sigma^2)$
parameters are constants	same π for every case	same μ, σ for every case
Deep	$X \mid c \sim \mathrm{Cat}\big(\pi_\theta(c)\big)$	$X \mid c \sim \mathcal{N}\big(\mu_\theta(c), \sigma_\theta(c)^2\big)$
parameters $= \mathrm{NN}_\theta(c)$	$\pi_\theta(c) = \mathrm{softmax}(\mathrm{NN}_\theta(c))$	$\mu_\theta = \mathrm{NN}, \;\; \sigma_\theta = \exp(\mathrm{NN})$
Training loss	$-\log \pi_{\theta,x}(c)$	$-\log \mathcal{N}\big(x \mid \mu_\theta(c), \sigma_\theta(c)^2\big)$
(per observed x)	cross-entropy (pmf)	Gaussian NLL (pdf)

We learn θ (the weights): one network gives a *different distribution for every c*.

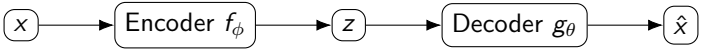
Autoencoder (AE): architecture

Autoencoders are deterministic reconstruction models (not probabilistic generative models).

- Encoder $f_\phi : \; x \mapsto z$
- Decoder $g_\theta : \; z \mapsto \hat{x}$
- Train to reconstruct: $\hat{x} \approx x$
- ℓ is a generic loss function

Typical objective

$$\min_{\theta,\phi} \sum_{n=1}^{N} \ell\Big(x^{(n)}, g_\theta(f_\phi(x^{(n)}))\Big)$$



Regularized Autoencoders: Three Ways to Stop Copying

The problem. A plain autoencoder minimizes $\|x - g_\theta(f_\phi(x))\|^2$. With enough capacity it can simply *copy* x through z without learning anything about the data. Each regularizer makes copying impossible or costly, so z must capture real structure.

Variant	**Change to training**	**What z is forced to learn**	**Example: handwritten digits**
Denoising	Input a corrupted $\tilde{x} = x +$ noise; target is the clean x: $\|x - g_\theta(f_\phi(\tilde{x}))\|^2$	What clean data looks like, so it can undo the noise	A smudged "3" is mapped back to a clean "3"
Sparse	Add a penalty $\lambda\|z\|_1$	To describe each input with only a few active units	A "3" activates a few units (e.g. two curves); the rest stay ≈ 0
Contractive	Add a penalty $\lambda\left\|\partial z/\partial x\right\|_F^2$	To ignore tiny changes in the input	A "3" shifted by one pixel gets almost the same z

Vocabulary you will see in papers: denoising AEs learn the *data manifold* (where real data lives), sparse AEs learn *parts-based* codes, and contractive AEs learn *local invariance*. These are names for the effects in the third column.

Why This Matters for the Bayesian View

Each penalty acts like a prior: a soft preference about codes.

- Sparse: $\lambda\|z\|_1 = -\log p(z) + \text{const}$ for a Laplace prior $p(z) \propto e^{-\lambda\|z\|_1}$ (as L2 weight decay $\leftrightarrow$ Gaussian prior).
- Contractive / denoising: prefer encoders that change slowly with x.

But two things are still missing:

1. **No distribution $p(z)$ to sample from.** A random z usually decodes to garbage, so we cannot generate new data.
2. **No uncertainty about the code.** $z = f_\phi(x)$ is a single point, even for a blurry or ambiguous input.

	Regularized AE	**VAE**
Prior on codes	implicit (penalty)	explicit $p(z) = \mathcal{N}(0, I)$
Encoder output	one point $z = f_\phi(x)$	a distribution $q_\phi(z \mid x) = \mathcal{N}\left(\mu_\phi(x), \sigma_\phi(x)^2\right)$
Generate new x?	no	yes: $z \sim p(z)$, then decode

The VAE encoder is a *distribution parametrized by a NN*, with x as input.

What Autoencoders Can and Cannot Do

Strengths (What AEs Do Well)

- **Reconstruction**: learn a compact code z that preserves information needed to recover x.
- **Compression**: dimensionality reduction via a learned encoder.
- **Useful representations**: good features for downstream tasks (classification, clustering).
- **Anomaly detection**: large reconstruction error flags unusual data.

For a broader perspective, see [1].

Limitations (What AEs Cannot Do)

- **Uncertainty**: decoder is deterministic; cannot express calibrated uncertainty about reconstructions.
- **Disentanglement**: latent factors rarely separate meaningfully unless explicit constraints or regularization are added.

Variational Autoencoder (VAE) — Overview

Key idea: Combine a *generative model* (decoder) with an *inference model* (encoder) and train them jointly on a single lower-bound objective (the ELBO).

Generative model (decoder)

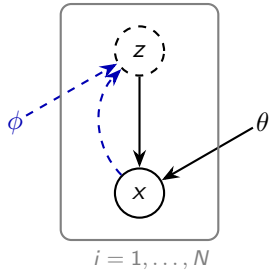
$$p_\theta(x, z) = \underbrace{p(z)}_{\text{prior}} \underbrace{p_\theta(x \mid z)}_{\text{NN decoder}}$$

$p(z) = \mathcal{N}(0, I)$; NN maps $z \to$ parameters of $p_\theta(x \mid z)$

Inference model (encoder)

$$q_\phi(z \mid x) = \mathcal{N}\big(\mu_\phi(x),\, \mathrm{diag}(\sigma_\phi(x)^2)\big)$$

NN maps $x \to (\mu, \sigma)$; approximates the true posterior $p_\theta(z \mid x)$



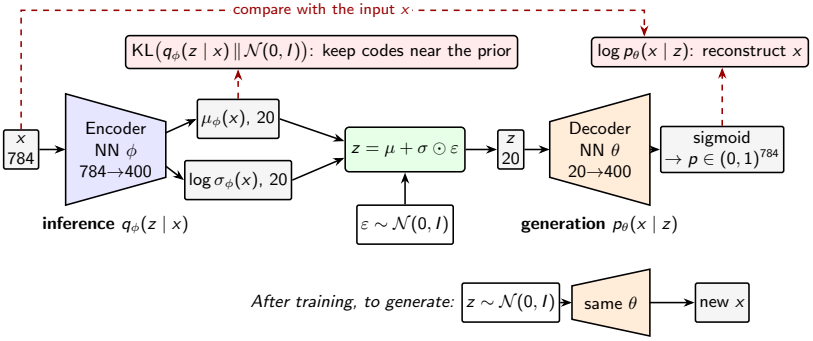
Solid: generative, $p(z)\, p_\theta(x \mid z)$
Dashed (blue): inference, $q_\phi(z \mid x)$
Each data point x_i has its own z_i;
θ, ϕ are shared by all points.

Core insight

Maximizing one objective (ELBO) trains both networks together.

VAE — Network Architecture

Example: MNIST digits, $x \in \{0,1\}^{784}$, latent $z \in \mathbb{R}^{20}$.



Training objective (maximize, over θ and ϕ jointly):

$$\mathrm{ELBO}(x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big]}_{\text{reconstruct } x \text{ well}} - \underbrace{\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}_{\text{stay close to the prior}}$$

The sampling step is written as $z = \mu + \sigma \odot \varepsilon$ (reparameterization) so that gradients can flow back into the encoder.

Goal of Unsupervised Learning: A Model $p(x \mid \theta)$ of the Data

Data: $x_1, \ldots, x_N$, no labels. The only thing we can fit a model to is the data itself, so we ask: *which θ makes the observed data most probable?*

$$\theta^* = \arg\max_\theta \sum_{i=1}^{N} \log p(x_i \mid \theta) \qquad \text{(maximum likelihood)}$$

Latent-variable model: each x is generated from a hidden code z:

$$z \sim p(z), \qquad x \sim p(x \mid z, \theta)$$

($p(x \mid z, \theta)$ is the decoder network).

We observe only x, never z, so z must be integrated out:

$$p(x \mid \theta) = \int p(z)\, p(x \mid z, \theta)\, dz.$$

What a good $p(x \mid \theta)$ gives us

- **Generation:** sample $z \sim p(z)$, decode to a new x.
- **Anomaly detection:** low $p(x \mid \theta)$ flags an unusual input.
- **Model comparison:** higher test log-likelihood is better.
- **Representation:** the posterior $p(z \mid x, \theta)$ gives a code for x.

We Never Compute $p(x \mid \theta)$: We Maximize a Lower Bound

For the encoder's distribution $q_\phi(z \mid x)$, the evidence splits exactly into two parts:

$$\log p(x \mid \theta) = \underbrace{\mathrm{ELBO}(\theta, \phi)}_{\text{computable}} + \underbrace{\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z \mid x, \theta)\big)}_{\geq 0,\ \text{the gap}} \quad \geq \quad \mathrm{ELBO}(\theta, \phi).$$

Maximizing the ELBO does two jobs at once:

- **over θ (decoder):** pushes $\log p(x \mid \theta)$ up. This is the actual goal: a good generative model.
- **over ϕ (encoder):** shrinks the gap, so $q_\phi(z \mid x)$ approaches the true posterior $p(z \mid x, \theta)$.

After training: estimating $\log p(x \mid \theta)$ when we need a number

Use q_ϕ as an importance-sampling proposal, with $z_k \sim q_\phi(z \mid x)$:

$$\log p(x \mid \theta) \approx \log \frac{1}{K} \sum_{k=1}^{K} \frac{p(z_k)\, p(x \mid z_k, \theta)}{q_\phi(z_k \mid x)}.$$

For model comparison or anomaly detection, not training. Biased low for small K; $K = 1$ gives the ELBO.

Reminder from the VI Lecture: Pathwise Gradient

VI lecture: variational distribution $q_\lambda(z)$ with parameters λ; maximize $\mathcal{L}(\lambda) = \mathbb{E}_{q_\lambda(z)}[f(z)]$, $f(z) = \log p(x, z) - \log q_\lambda(z)$.

Problem: the distribution depends on λ, but a sampled value $z^{(s)}$ does not, so naively $\partial f(z^{(s)})/\partial\lambda = 0$ (e.g. $\mathbb{E}[z] = \lambda$ has gradient 1, not 0).

Fix (reparameterization): move the randomness into noise that does not depend on λ:

$$z = g(\lambda, \epsilon), \quad \epsilon \sim p(\epsilon) \quad \Longrightarrow \quad \mathbb{E}_{q_\lambda}\big[f(z)\big] = \mathbb{E}_\epsilon\big[f\big(g(\lambda, \epsilon)\big)\big].$$

Pathwise estimator

$$\nabla_\lambda \mathbb{E}_{q_\lambda}[f(z)] = \mathbb{E}_\epsilon\Big[\nabla_z f(z)\big|_{z=g(\lambda,\epsilon)}\ \nabla_\lambda g(\lambda, \epsilon)\Big] \approx \frac{1}{S}\sum_{s=1}^{S} \nabla_z f\big(z^{(s)}\big)\ \nabla_\lambda g\big(\lambda, \epsilon^{(s)}\big)$$

Gaussian case: $\lambda = (\mu, \sigma)$, $z = g(\lambda, \epsilon) = \mu + \sigma\epsilon$, $\epsilon \sim \mathcal{N}(0, 1)$, so $\nabla_\mu g = 1$ and $\nabla_\sigma g = \epsilon$.

VAE: The Same Idea with Encoder ϕ and Decoder θ

What changes: the parameters of q are no longer learned directly. The encoder network computes them for each x:

$$q_\phi(z \mid x) = \mathcal{N}\big(\mu_\phi(x),\ \mathrm{diag}(\sigma_\phi(x)^2)\big).$$

We learn the encoder weights ϕ together with the decoder weights θ by maximizing

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{q_\phi(z|x)}\big[f(z)\big] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big), \qquad f(z) = \log p_\theta(x \mid z).$$

∇_θ: easy

q_ϕ does not depend on θ, so plain Monte Carlo works:

$$\nabla_\theta \mathcal{L} \approx \frac{1}{S} \sum_s \nabla_\theta f\big(z^{(s)}\big)$$

with $z^{(s)} \sim q_\phi(z \mid x)$.

∇_ϕ: reparameterize

Same problem as in VI, same fix:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

$$\nabla_\phi \,\mathbb{E}[f] = \mathbb{E}_\epsilon\Big[\Big(\frac{\partial z}{\partial \phi}\Big)^\top \nabla_z f(z)\Big]$$

Pathwise Gradient for the Gaussian Encoder

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \qquad \frac{\partial z_j}{\partial \mu_j} = 1, \quad \frac{\partial z_j}{\partial \sigma_j} = \epsilon_j.$$

Step 1: gradients with respect to the encoder outputs

$$\nabla_\mu \mathbb{E}[f] \approx \frac{1}{S} \sum_s \nabla_z f\big(z^{(s)}\big), \qquad \nabla_\sigma \mathbb{E}[f] \approx \frac{1}{S} \sum_s \nabla_z f\big(z^{(s)}\big) \odot \epsilon^{(s)}$$

Step 2: backpropagate into the encoder weights

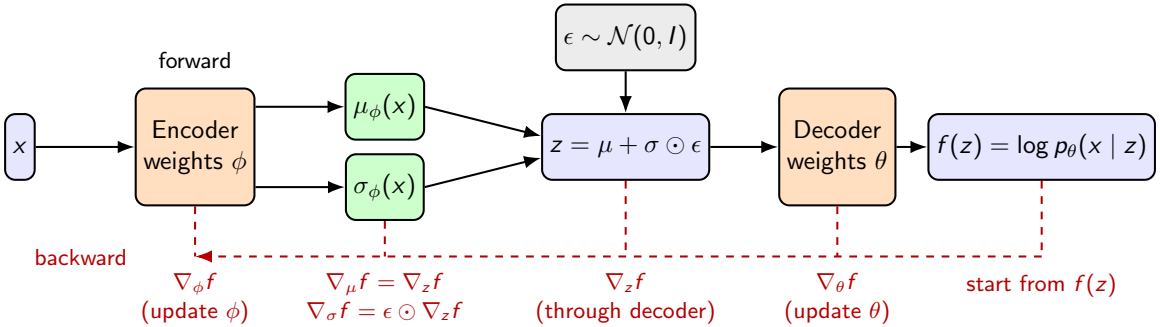
$$\nabla_\phi \mathbb{E}[f] = \Big(\frac{\partial \mu_\phi(x)}{\partial \phi}\Big)^\top \nabla_\mu \mathbb{E}[f] + \Big(\frac{\partial \sigma_\phi(x)}{\partial \phi}\Big)^\top \nabla_\sigma \mathbb{E}[f]$$

In practice ($S = 1$): draw $\epsilon \sim \mathcal{N}(0, I)$ and compute

$$\widehat{\mathcal{L}} = \log p_\theta\big(x \mid \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon\big) - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big).$$

Autodiff gives $\nabla_\theta \widehat{\mathcal{L}}$ and $\nabla_\phi \widehat{\mathcal{L}}$ in one backward pass.

Forward vs. Backward Flow in the VAE (Reparameterization Trick)



Chain rule, step by step (per dimension $\partial z_j/\partial \mu_j = 1$, $\partial z_j/\partial \sigma_j = \epsilon_j$):

$$\nabla_\phi f = \Big(\frac{\partial \mu_\phi(x)}{\partial \phi}\Big)^\top \underbrace{\nabla_z f}_{\nabla_\mu f} + \Big(\frac{\partial \sigma_\phi(x)}{\partial \phi}\Big)^\top \underbrace{(\epsilon \odot \nabla_z f)}_{\nabla_\sigma f}$$

$\nabla_z f$ is only an intermediate quantity; the encoder is updated with ∇_ϕ. The KL term adds its own gradient directly at (μ, σ); it does not pass through z.

Analytical KL for Gaussian Prior & Posterior

When $q_\phi(z|x) = \mathcal{N}(\mu, \mathrm{diag}(\sigma^2))$ and $p(z) = \mathcal{N}(0, I)$, the KL is available in closed form — **no sampling needed for this term**.

Derivation (scalar case $d = 1$):

$$D_{\mathrm{KL}}(q\|p) = \int q(z) \log \frac{q(z)}{p(z)}\, dz = \mathbb{E}_q\left[\log \frac{\mathcal{N}(\mu, \sigma^2)}{\mathcal{N}(0,1)}\right]$$
$$= \mathbb{E}_q\left[-\tfrac{(z-\mu)^2}{2\sigma^2} + \tfrac{z^2}{2} + \tfrac{1}{2}\log\sigma^{-2}\right] = \tfrac{1}{2}\left(\sigma^2 + \mu^2 - 1 - \log\sigma^2\right)$$

General d-dimensional diagonal Gaussian

$$\boxed{D_{\mathrm{KL}}(q_\phi\|p) = \frac{1}{2}\sum_{j=1}^{d}\left(\sigma_j^2 + \mu_j^2 - 1 - \log\sigma_j^2\right) \geq 0}$$

Sanity checks: μ=0, σ=1: $D_{\mathrm{KL}} = \frac{1}{2}(1+0-1-0) = 0$. ✓ μ=1, σ=1: $D_{\mathrm{KL}} = \frac{1}{2}(1+1-1-0) = \frac{1}{2}$. ✓

In practice ($S = 1$): draw $\varepsilon \sim \mathcal{N}(0, I)$ and compute

$$\widehat{\mathcal{L}} = \log p_\theta\left(x \mid \mu + \sigma \odot \varepsilon\right) - \tfrac{1}{2}\textstyle\sum_j \left(\mu_j^2 + \sigma_j^2 - 1 - \log\sigma_j^2\right),$$

with $(\mu, \sigma) = (\mu_\phi(x), \sigma_\phi(x))$. Autodiff gives $\nabla_\theta \widehat{\mathcal{L}}$ and $\nabla_\phi \widehat{\mathcal{L}}$ in one backward pass.

Training the VAE: AEVB / Reparameterization Algorithm

Algorithm (Kingma & Welling, 2013)

Inputs: dataset $\mathcal{D}$; encoder $q_\phi(z|x)$; decoder $p_\theta(x|z)$

Initialize θ, ϕ randomly.

Repeat until convergence:

1. Sample mini-batch $\mathcal{M} \subset \mathcal{D}$
2. For each $x \in \mathcal{M}$:
   1. Compute encoder outputs $(\mu_\phi(x), \sigma_\phi(x))$
   2. Sample $\epsilon \sim \mathcal{N}(0, I)$
   3. Reparameterize $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$
   4. Compute reconstruction log-likelihood $\log p_\theta(x|z)$ via the decoder
   5. Compute analytic KL: $\mathrm{KL}(q_\phi(z|x)\|p(z))$
3. Form Monte Carlo ELBO estimate:

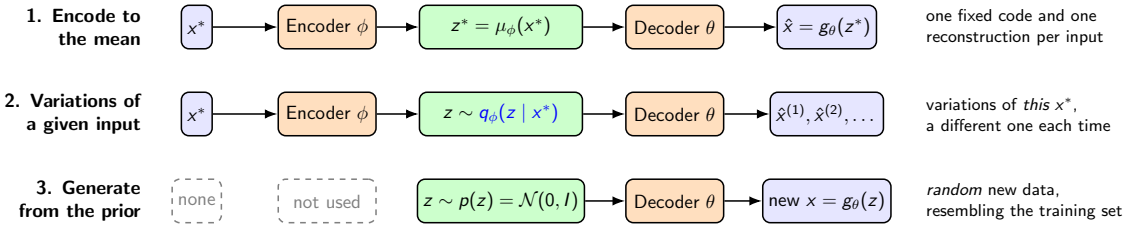
$$\widetilde{\mathcal{L}}(x) = \log p_\theta(x|z) - \mathrm{KL}(q_\phi(z|x)\|p(z))$$

4. Update parameters via stochastic gradient:

$$\theta \leftarrow \theta + \eta \nabla_\theta \widetilde{\mathcal{L}}, \qquad \phi \leftarrow \phi + \eta \nabla_\phi \widetilde{\mathcal{L}}.$$

Using the Trained VAE: Three Modes

Notation: $g_\theta(z)$ is the decoder network's output, i.e. the mean of $p_\theta(x \mid z)$. We usually show $g_\theta(z)$ rather than sampling $x \sim p_\theta(x \mid z)$, which only adds observation noise.



- **Mode 1** is the same computation as a regular autoencoder. The difference comes from training (noisy codes and the KL term), which gives a smooth, organized latent space.
- **Mode 2 vs. mode 3:** variations of *this* input vs. *random* new data. Mode 3 needs no input; it works because the KL term pulled every $q_\phi(z \mid x)$ toward $\mathcal{N}(0, I)$.

Using the Trained VAE: Applications of Each Mode

Tags show which networks are needed *at use time*: **[E]** encoder, **[D]** decoder.

Mode	Applications
1. Encode to the mean $z^* = \mu_\phi(x^*)$	**Features / embeddings [E]:** clustering, classifier inputs, 2-D visualization (decoder needed only for training). **Compression [E at sender, D at receiver]:** store z^* (20 numbers instead of 784), decode later. **Denoising [E+D]:** the output $g_\theta(\mu_\phi(\tilde{x}))$ looks like clean training data. **Anomaly detection [E+D]:** large reconstruction error or low ELBO flags an unusual input (faulty sensor signal, defective part).
2. Variations of a given input $z \sim q_\phi(z \mid x^*)$	**Label-preserving augmentation [E+D]:** variants of a labeled (x^*, y) stay near its code, so they keep label y; can target rare cases (fault examples). **Per-input uncertainty [E]:** a large $\sigma_\phi(x^*)$ flags ambiguous inputs (a "4" or a "9"?). **Completion [E+D]:** several plausible fill-ins of a missing part of x^*. **Estimating** $\log p(x^* \mid \theta)$ **[E+D]:** importance sampling with $z_k \sim q_\phi(z \mid x^*)$.
3. Generate from the prior $z \sim \mathcal{N}(0, I)$	**Synthetic data [D]:** extra or shareable data; labels unknown unless a conditional VAE $p_\theta(x \mid z, y)$ is used. **Exploring the latent space [D]:** change one coordinate of z, see what changes in x. **Interpolation [E+D]:** decode points between the codes of two inputs.

Using the Trained VAE: Limitations of Each Mode

1. Encode to the mean

Blurrier than a regular autoencoder's reconstructions (the price of an organized latent space). The single code $\mu_\phi(x^*)$ ignores the encoder's uncertainty.

2. Variations of a given input: the uncertainty is approximate

$\sigma_\phi(x^*)$ is the spread of an *approximate* posterior:

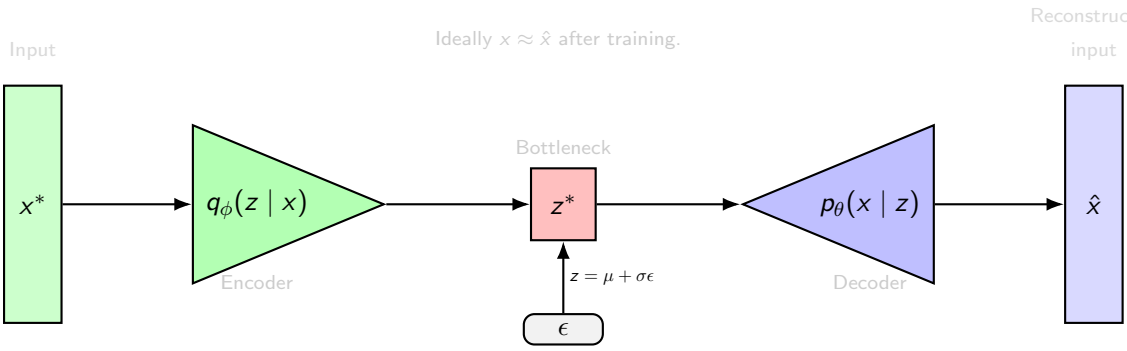
- reverse KL with a diagonal Gaussian tends to **underestimate** the spread (VI lecture);
- one encoder predicts q for every x, which adds error (the amortization gap);
- uncertainty about the weights ϕ, θ is not included.

Useful as a signal, but not calibrated uncertainty.

3. Generate from the prior

You cannot choose which class you get (without a conditional VAE). Regions of $\mathcal{N}(0, I)$ not covered by any $q_\phi(z \mid x)$ decode to unrealistic, often blurry samples.

Encoder–Decoder Flow After Training



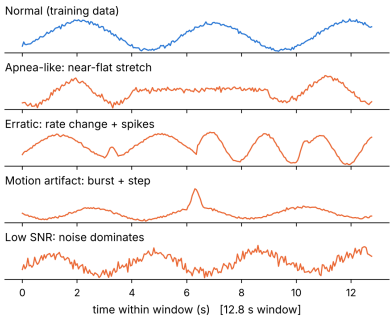
Encoder: inference (features, uncertainty). Decoder: generation (reconstruction, new samples).

Case Study: Detecting Abnormal Breathing with a VAE

Task. A wearable sensor (e.g. a chest belt) records breathing. Flag the short stretches where breathing is abnormal.

Difficulty. Abnormal events are rare and varied, so we cannot collect labeled examples of every kind.

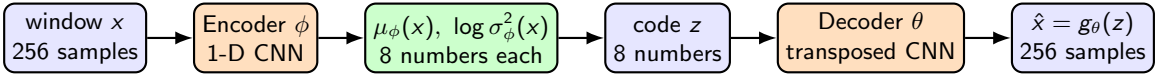
Idea. Learn what *normal* breathing looks like, and flag whatever the model cannot explain.



Data (synthetic; from the notebook)

Signal at 20 Hz, cut into windows of 256 samples (12.8 s, about 2–4 breaths). **Each window is one data point** $x \in \mathbb{R}^{256}$, normalized to zero mean and unit variance.
Train on 4000 **normal** windows only; test on 1000 held-out normal and 4×800 abnormal windows.

The Model and the Anomaly Score



Training on normal windows only

Gaussian decoder with fixed variance, $p_\theta(x \mid z) = \mathcal{N}\big(g_\theta(z), \sigma^2 I\big)$, so the reconstruction term is the squared error. Loss (roughly a negative ELBO):

$$\|x - \hat{x}\|^2 + \beta\, \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, \mathcal{N}(0, I)\big), \qquad \beta = 0.2$$

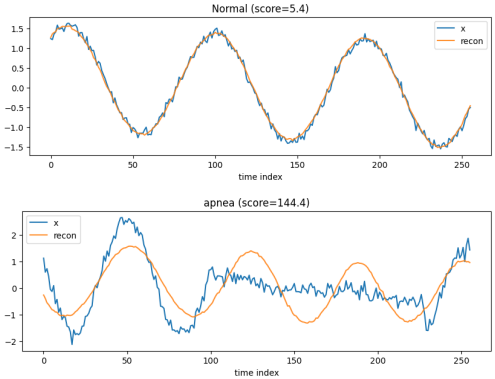
(β-VAE: $\beta < 1$ weakens the pull toward the prior and gives sharper reconstructions.)

Anomaly score at test time: encode to the mean (mode 1)

$$S(x) = \underbrace{\big\|x - g_\theta\big(\mu_\phi(x)\big)\big\|^2}_{\text{cannot rebuild } x} + \beta\, \underbrace{\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}_{\text{unusual code}}$$

Flag x if $S(x)$ exceeds the 99th percentile of S on held-out normal windows ($\approx$1% false alarms).

Results: Why It Works, and What to Watch Out For



The decoder only knows normal breathing: during the apnea it "fills in" breaths that are not there, so the reconstruction error is large. (Best-scoring normal and worst-scoring apnea window.)

Results on the test set

False alarms on normal windows: 1.0%
Abnormal windows detected: 99.6%

apnea	erratic	motion	low SNR
100%	98.4%	100%	99.9%

What drives detection: mostly the reconstruction term; the KL values of normal and abnormal windows overlap much more. *Erratic* breathing is hardest, because part of each window still looks normal.

Caveats

Synthetic data: real signals vary across people, sensor placement and activity, so expect weaker separation. Per-window normalization removes absolute amplitude. Windows are scored independently; dependence between consecutive windows is ignored.

Practical Concern: Posterior Collapse

The KL term is computed *for each data point*. Averaged over the data it splits into two parts, with $q_\phi(z) = \mathbb{E}_x\big[q_\phi(z \mid x)\big]$ the distribution of all codes together:

$$\mathbb{E}_x\Big[\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)\Big] = \underbrace{I_q(x; z)}_{\text{information } z \text{ carries about } x} + \underbrace{\mathrm{KL}\big(q_\phi(z) \,\|\, p(z)\big)}_{\text{all codes together vs. } \mathcal{N}(0,I)}$$

What we want

The **cloud of all codes** looks like $\mathcal{N}(0, I)$ (second term small), so sampling from the prior works.

Each **individual** $q_\phi(z \mid x)$ differs from the prior: different inputs need different codes (first term > 0).

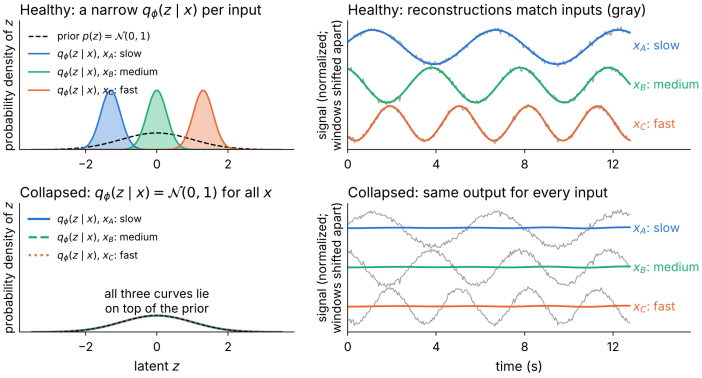
Posterior collapse

$q_\phi(z \mid x) \approx p(z)$ for *every* x.

The KL is ≈ 0, but z carries no information about x ($I_q(x; z) \approx 0$), and the decoder ignores z: a Gaussian decoder outputs roughly the average x.

Key point: the KL term *penalizes information*. Every bit that z carries about x costs KL; reconstruction pays it back only if z actually helps rebuild x.

Posterior Collapse: A Picture



How can $q_\phi(z \mid x) = p(z)$ for every x? The encoder is just a function of x. It can output the *same* numbers, $\mu_\phi(x) = 0$ and $\log \sigma^2_\phi(x) = 0$, for every input (e.g. if the weights of its last layer go to zero). Then every code is simply a draw $z \sim p(z)$, from the *same* distribution whatever the input. Seeing z tells the decoder nothing about which window it came from, so the best it can do is output the average window.

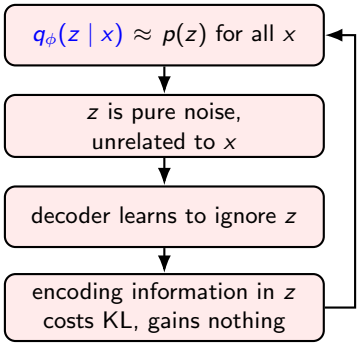
Why Posterior Collapse Happens

Early in training the two terms pull differently:

- the KL gradient is immediate: shrinking $\mu_\phi \to 0$, $\sigma_\phi \to 1$ lowers the loss right away;
- the reconstruction gain from an informative z is small and noisy, because the decoder has *not yet learned to use* z.

So the KL is reduced first, and every $q_\phi(z \mid x)$ moves toward the prior.

Why it gets stuck (a local optimum)



Detecting and Preventing Posterior Collapse

Detect: average the KL of each latent dimension j over the data, $\overline{\mathrm{KL}_j}$. A value ≈ 0 means dimension j has collapsed. Often only some collapse (*partial* collapse); if all do, reconstructions ignore the input.

Fix: KL annealing

$$\mathcal{L}_t = \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] - \beta_t \,\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big), \qquad \beta_t : 0 \nearrow 1$$

Start with $\beta_t \approx 0$: information in z is (almost) free, so the decoder learns to use z. Then increase β_t gradually to 1. By the time the full KL cost applies, z is useful enough to be worth it.

Not the same as a β-VAE with a fixed β

A fixed $\beta > 1$ (for disentangled codes) makes collapse *more* likely; a fixed $\beta < 1$ (e.g. $\beta = 0.2$ in the breathing example) makes it less likely. Annealing uses $\beta < 1$ only temporarily and ends at the true ELBO ($\beta = 1$).

Practical Recommendations

Architecture

- **Encoder**: MLP or CNN; output $(\mu_\phi, \log \sigma_\phi)$ — use log-parameterisation to ensure $\sigma > 0$
- **Decoder**: match to data type — Gaussian MSE for continuous signals, Bernoulli BCE for binary
- **Latent dim**: start small (8–32); too large $\Rightarrow$ many unused dimensions (partial collapse)

Training

- **Optimiser**: Adam (lr $\approx 10^{-3}$, decay to 10^{-4})
- **Batch size**: ≥ 64; larger batches $\Rightarrow$ more stable KL estimates
- **KL annealing**: linear or cyclical schedule; avoid switching too fast
- **Warm-up**: 10–50 epochs of reconstruction-only training before enabling KL

Monitoring during training

Track **both** reconstruction loss and KL separately. Healthy training: KL rises gradually, reconstruction falls. Collapse sign: KL $\to 0$ and reconstruction stagnates. Also track the average KL **per latent dimension**: ≈ 0 means that dimension is unused.

Full Gaussian-VAE ELBO

Gaussian decoder assumption, with decoder mean $g_\theta(z)$ and fixed variance σ^2:

$$p_\theta(x \mid z) = \mathcal{N}\big(x;\, g_\theta(z), \sigma^2 I\big).$$

ELBO (maximized):

$$\mathcal{L}(x;\theta,\phi) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \mathrm{KL}(q_\phi(z \mid x)\|p(z)).$$

Gaussian log-likelihood:

$$\mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)] = -\frac{1}{2\sigma^2}\mathbb{E}_{q_\phi}\|x - g_\theta(z)\|^2 + \text{const.}$$

KL term:

$$\mathrm{KL}(q_\phi(z \mid x)\|p(z)) = \frac{1}{2}\sum_i \big(\mu_i^2 + \sigma_i^2 - 1 - \log\sigma_i^2\big).$$

Loss (minimized in practice): multiply $-\mathcal{L}$ by $2\sigma^2$ and drop constants,

$$\boxed{J(x) = \|x - g_\theta(z)\|^2 + \beta\,\mathrm{KL}\big(q_\phi(z \mid x)\|p(z)\big), \qquad \beta = 2\sigma^2.}$$

$\beta = 1$ corresponds to $\sigma^2 = \frac{1}{2}$. Choosing another β (the β-VAE) is equivalent to assuming a different decoder variance. In the breathing example, $\beta = 0.2$ with a sum-of-squares reconstruction corresponds to $\sigma^2 = 0.1$.

VAE: Advantages & Limitations

Advantages

- End-to-end backprop (single ELBO objective)
- Fast amortised inference at test time
- Structured, continuous latent space
- Works for any data type with a suitable decoder
- Principled anomaly scoring via ELBO

Limitations

- ELBO is only a *lower bound* on $\log p(x)$
- Posterior collapse (decoder too powerful)
- Gaussian posterior may be too simple (use normalising flows for richer q)
- Generated samples often blurry (mode averaging)

Rule of thumb for engineering applications

If you need **anomaly detection** or **representation learning** on sensor data: VAE is an excellent baseline. If you need **photo-realistic generation**: consider more advanced models.

Summary

What we covered

1. **NN as distribution parametriser** – NNs map inputs to parameters of probabilistic models.
2. **Autoencoders** – deterministic codes; regularizers stop copying, but there is no $p(z)$ to sample from and no uncertainty about the code.
3. **ELBO** – $p(x \mid \theta)$ is intractable, so we maximize a lower bound: over θ it improves the model, over ϕ it tightens the bound.
4. **VAE** – an amortized encoder plus the reparametrisation trick enable end-to-end backprop for continuous latent variables.
5. **Using a trained VAE** – encode to the mean, variations of a given input, generation from the prior (e.g. breathing anomaly detection).
6. **Posterior collapse** – detect it with the KL per dimension; prevent it with KL annealing.

Further reading: Kingma & Welling (2013); Rezende et al. (2014)

References I


Bank, Dor, Noam Koenigstein, and Raja Giryes (2020). *Autoencoders*. arXiv:2003.05991.